

MethylPipeline Comprehensive Analysis

Theory, architecture, implementation, and usage

MethylPipeline Team

2026-05-29

Table of contents

1	Executive Summary	2
2	Scope and sources	2
3	Theory	3
3.1	Data representation: methylation values and centroid summaries	3
3.2	ECDF reconstruction and nonparametric two-sample screening	3
3.3	Canonical biological effect size	3
3.4	Stability selection (Monte Carlo)	4
3.5	Gene-level importance theory	4
3.6	Disease progression synthesis theory	4
3.7	Method taxonomy and caveats	4
4	Architecture	5
4.1	Package architecture	5
4.2	End-to-end data flow	5
4.3	Core phase orchestration	5
4.4	Canonical data contracts	6
5	Implementation	6
5.1	Mapper implementation highlights	6
5.2	Validation configuration and model-bundle implementation	6
5.3	Enricher strict-contract behavior	7
5.4	Progression robustness behavior	7
5.5	GPU and fallback implementation	7
5.6	gene_importance contract across the pipeline	7
6	Usage	7
6.1	Environment and installation	7
6.2	Project JSON configuration model	8
6.3	Staged operational workflow	8
6.4	Freeze phase diagram	9

7 Equations and score decomposition figure	10
8 Limitations and operational caveats	10
9 Rendering	10

1 Executive Summary

MethylPipeline is a monorepo for DNA methylation analysis that combines:

1. empirical distribution modeling (ECDF/histogram centroids),
2. differential methylation discovery with FDR control,
3. stability-based feature selection via Monte Carlo,
4. genomic interpretation (mapping + enrichment + progression synthesis), and
5. model training/selection workflows.

The scientific spine is:

- methyl-validation (orchestration) -> methyl-centroid -> methyl-detector -> methyl-mapper -> methyl-enricher -> methyl-disease-progression.

Supporting packages (methylclassifier, methylpredictor, methylalignmentqc, methylcluster) integrate around this core.

2 Scope and sources

This analysis synthesizes repository docs and implementation modules, with emphasis on core scientific flow and data contracts.

Primary references include:

- docs/MethylPipeline-overview.qmd
- docs/theory/chapters/*.qmd
- packages/*/docs/{THEORY,IMPLEMENTATION,USAGE}.md
- packages/methylutils/methyl_utils/statistical_tests.py
- packages/methylvalidation/methyl_validation/{cli.py,pipeline_runner.py,stability.py,model_
- packages/methylmapper/methyl_mapper/bedtools_mapper.py
- packages/methylenricher/methyl_enricher/{enricher.py,ensure_complete.py,config.py}
- packages/methyldiseaseprogression/methyl_disease_progression/{progression.py,gene_set_cove

3 Theory

3.1 Data representation: methylation values and centroid summaries

At locus i for sample s :

$$x_{si} = \frac{m_{si}}{m_{si} + u_{si}},$$

where m and u are methylated/unmethylated counts.

Cohort sufficient statistics used throughout the pipeline:

$$\hat{\mu}_i = \frac{S_{x,i}}{N_i}, \quad \hat{\sigma}_i^2 = \max\left(\frac{S_{x^2,i} - S_{x,i}^2/N_i}{\max(N_i - 1, 1)}, \varepsilon\right).$$

`methylcentroid` stores these plus binned histograms per locus/context/chromosome, enabling downstream distribution comparison without reloading raw read-level data.

3.2 ECDF reconstruction and nonparametric two-sample screening

The ECDF at bin edges is reconstructed from centroid histograms and interpolated (PCHIP-style) in `methylutils`.

Core screening includes:

- Kolmogorov-Smirnov style distance on reconstructed ECDFs,
- Mann-Whitney U approximation from bin counts,
- Storey q-values for multiple testing control.

Storey components (implemented in `storey_qvalues`):

$$\hat{\pi}_0(\lambda) = \frac{\#\{p_j > \lambda\}}{(1 - \lambda)m}, \quad q_{(i)} = \min_{j \geq i} \frac{\hat{\pi}_0 m p_{(j)}}{j}.$$

3.3 Canonical biological effect size

MethylPipeline separates statistical significance from biological ranking. The canonical effect-size formulation in `effect_size_from_components` is:

$$\text{effect_size}_i = |\Delta\mu_i| (1 - \text{overlap}_i) \exp\left(-\lambda_{\text{var}}(\sqrt{\sigma_{1i}^2} + \sqrt{\sigma_{2i}^2})\right),$$

optionally multiplied by a mean-level weighting factor.

This is a prioritization score, not a p-value.

3.4 Stability selection (Monte Carlo)

With repeated train/validation splits, recurrence of each discovered DMP is:

$$\hat{f}(d) = \frac{1}{R^*} \sum_{r=1}^{R^*} \mathbf{1}[d \in \mathcal{D}_r].$$

Stable panel construction in `stability.py` ranks by:

$$\text{combined_score}(d) = \text{effect_size}(d) \sqrt{\hat{f}(d)},$$

followed by thresholding and optional elbow trimming.

3.5 Gene-level importance theory

In mapper aggregation, the canonical gene importance is tied to effect consistency and support:

$$\text{gene_effect_compound} = \text{gene_effect_abs_wmean} \times \text{gene_direction_coherence} \times \sqrt{\text{gene_support_freq}}.$$

`gene_importance` is canonicalized to this compound metric in current implementation.

3.6 Disease progression synthesis theory

`methyldiseaseprogression` is intentionally deterministic and file-driven:

- input: per-stage mapper and enricher outputs,
- output: ranked long tables and heuristic stage labels (e.g., `early_only`, `late_only`, `monotonic_up`, `stable_across_stages`).

This is evidence synthesis over staged contrasts, not a latent-time probabilistic model.

3.7 Method taxonomy and caveats

- Principled/statistical: FDR control, centroid sufficient statistics, asymptotic tests.
- Approximate: histogram-derived MWU and ECDF grid approximations.
- Heuristic: effect-size weighting, stability cutoffs, progression labels.
- External dependencies: Enrichr services, optional STRING/PPI data.

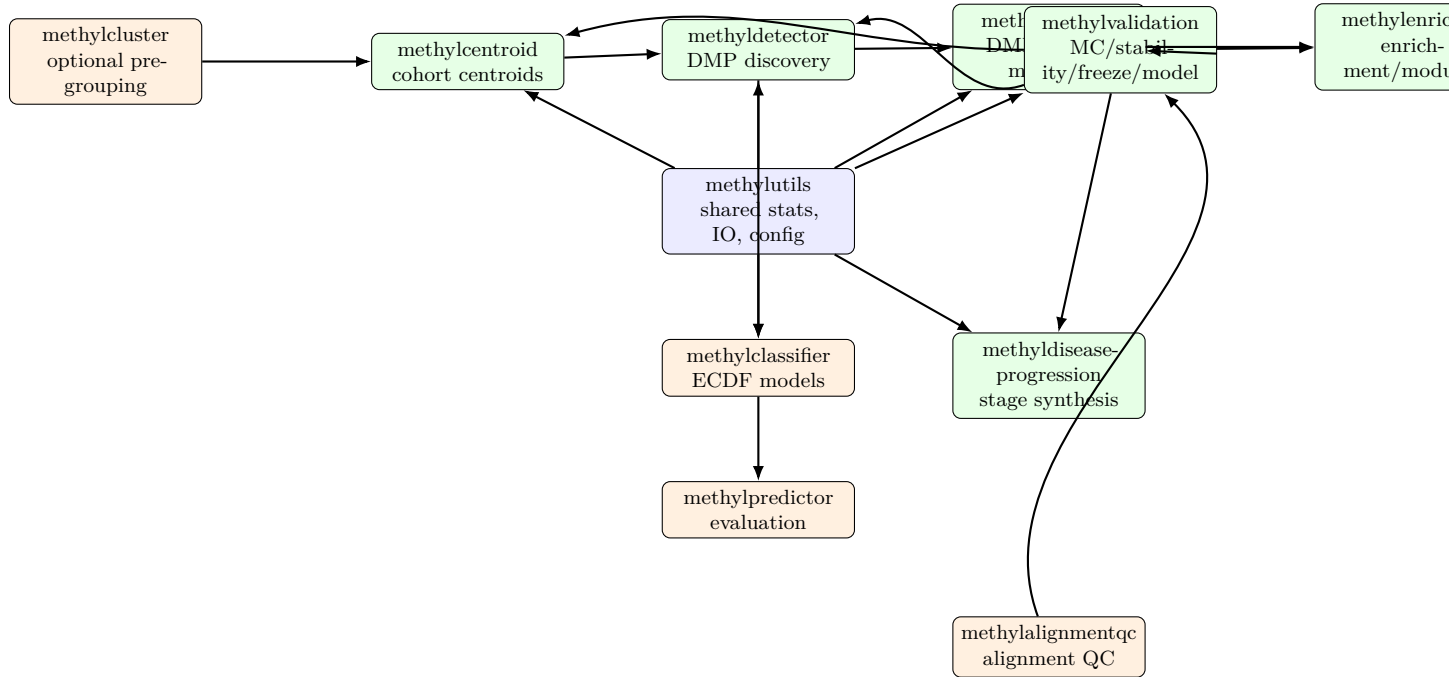


Figure 1: High-level package architecture and primary dependencies in MethylPipeline.

4 Architecture

4.1 Package architecture

The monorepo package graph centers on shared `methylutils` and orchestration via `methylvalidation`.

4.2 End-to-end data flow

The data flow from samples to model/paper outputs is phase-based.

4.3 Core phase orchestration

`methyl-validation` in `pipeline_runner.py` orchestrates:

- Monte Carlo iterations (`centroid -> detector` only),
- stability aggregation (`--stability`),
- production freeze (`--freeze`):
 - fixed DMP panel injection,
 - production centroid/detector/mapper/enricher/progression pipeline,
- model training/evaluation (`--model`, `--model-mc`, `--post-model-validation`).

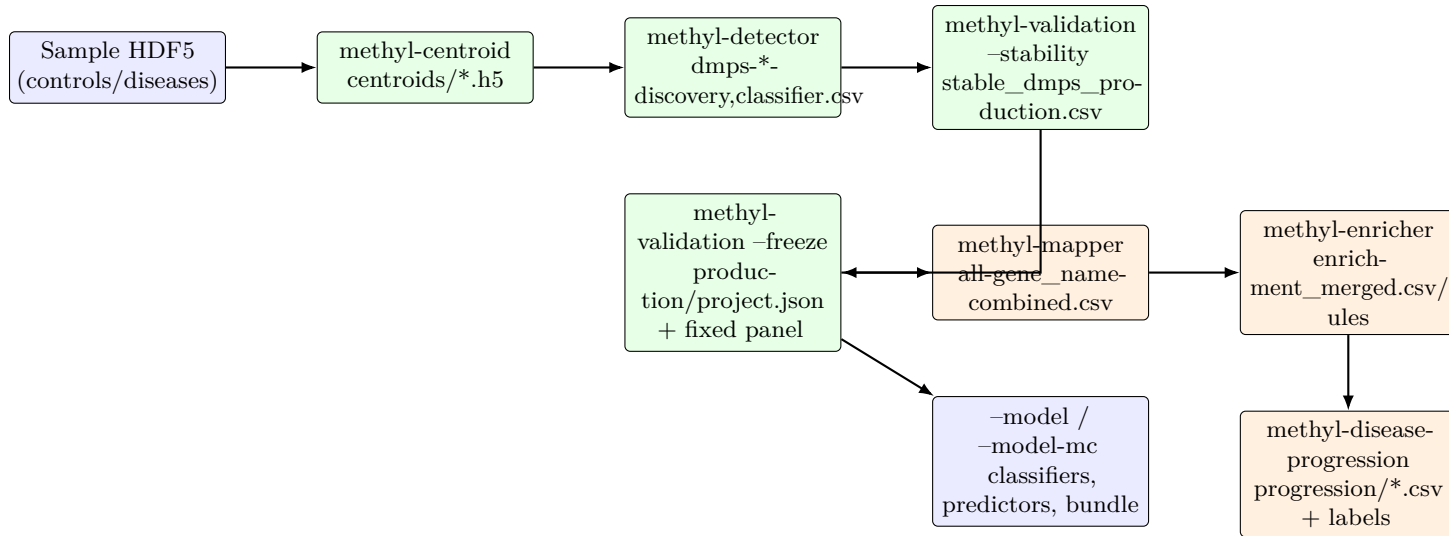


Figure 2: Operational data flow across stability, freeze, enrichment, progression, and model stages.

4.4 Canonical data contracts

Key contracts between stages:

- detector exports DMP panels (`dmps-*.csv`),
- mapper exports per-comparison `all-gene_name-combined.csv`,
- enricher consumes mapper combined CSV and emits `enrichment_merged.csv` and module tables,
- progression consumes mapper+enricher stage outputs,
- model bundle uses detector DMP index plus optional mapper annotation cache.

5 Implementation

5.1 Mapper implementation highlights

In `bedtools_mapper.py`:

- explicit detailed-to-parent feature roll-up map (`_DETAILED_TO_PARENT`),
- normalization path (`_normalize_feature_type`) preserving explicit features when configured,
- exclusive (gene, DMP) hit assignment for feature-level counting,
- combined p/q and weighted effect metrics,
- canonical `gene_importance` assignment from `gene_effect_compound`,
- curated output column pruning in `_prune_gene_output_columns`.

5.2 Validation configuration and model-bundle implementation

In `methyl_validation/config.py`:

- strict Pydantic models for MC + backend profiles,
- `DEFAULT_MAPPER_GENE_COLUMNS` and configurable `mapper_gene_columns`.

In `methyl_validation/model_bundle.py`:

- `build_mapper_annotation_cache` composes per-locus mapper annotation rows,
- optional join of configured per-gene mapper columns,
- HDF5 writing/reading in bundle schema v2 with core and passthrough columns.

5.3 Enricher strict-contract behavior

`methyl_enricher/enricher.py` enforces mapper contract fields and canonical score usage:

- required columns include `gene_name`, `gene_importance`, `mean_effect_size`, `unique_dmps`,
- strict filter checks fail with explicit `ValueError` when required columns are absent.

Compatibility bridge implemented for legacy filter knob:

- if `min_dmp_count` requested and `dmp_count` missing,
- fallback applies threshold to `unique_dmps` with warning.

5.4 Progression robustness behavior

`progression.py` and `gene_set_coverage.py` now support per-stage graceful degradation:

- malformed/missing stage inputs are logged and skipped in stage loops,
- global pipeline can continue in non-strict mode,
- strict mode still fails fast when required by caller.

5.5 GPU and fallback implementation

`methyl_utils/gpu_detection.py` centralizes GPU capability checks and state.

`methyl_utils/statistical_tests.py` includes guarded GPU execution with CPU fallback in critical statistics paths (e.g., Storey q-values and Beta MOM estimation) so transient CUDA runtime issues do not crash full runs.

5.6 gene_importance contract across the pipeline

6 Usage

6.1 Environment and installation

Preferred host setup from repo root:

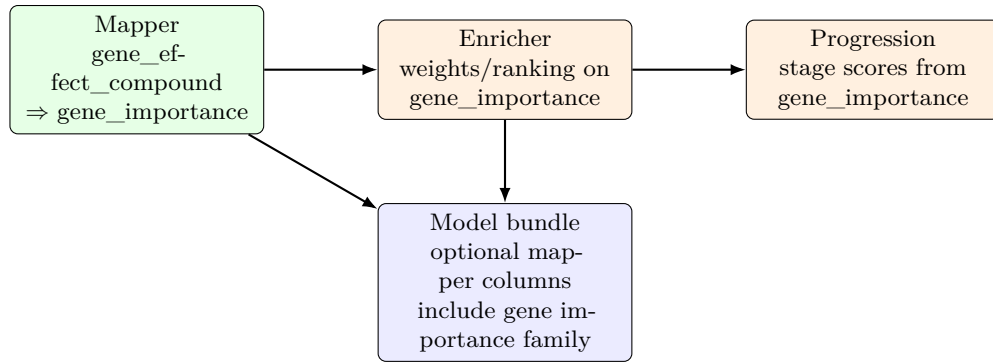


Figure 3: Canonical gene_importance contract from mapper to downstream components.

```
cd /home/ubuntu/MethylPipeline
bash scripts/setup_host.sh --system-deps --gpu
source .venv/bin/activate
```

For explicit GPU profile alignment:

```
source .venv/bin/activate
pip install -r requirements-gpu-cuda12.txt \
  -c scripts/constraints-cuda12.txt \
  --extra-index-url https://pypi.nvidia.com
```

6.2 Project JSON configuration model

Core structure:

- top-level project metadata (project_name, output_base, samples_base_path, chromosome/context scope),
- cohort hierarchy (controls, diseases, stage groups),
- comparison strategy (control_vs_each_disease, all_pairs, or explicit list),
- step_config blocks:
 - centroid, detection, mapper, enricher, progression, validation, optional others.

Reference docs:

- docs/theory/chapters/11-project-configuration.qmd
- docs/theory/chapters/13-configuration-reference.qmd

6.3 Staged operational workflow

Typical commands:

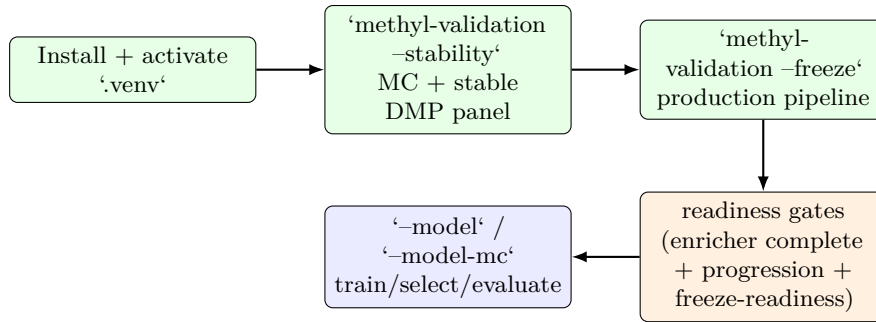


Figure 4: Recommended operational sequence for production-grade runs.

```

# 1) Stability
methyl-validation --project /work/prostate-cancer/configs/project_Healthy_vs_PCa1-4-CG.json --stability

# 2) Freeze
methyl-validation --project /work/prostate-cancer/configs/project_Healthy_vs_PCa1-4-CG.json --freeze

# 3) Model
methyl-validation --project /work/prostate-cancer/configs/project_Healthy_vs_PCa1-4-CG.json --model
  
```

Important behavior:

- `--skip-detection` is supported in `--stability` and `--freeze`,
- `--skip-detection` implies `--skip-centroid`.

6.4 Freeze phase diagram

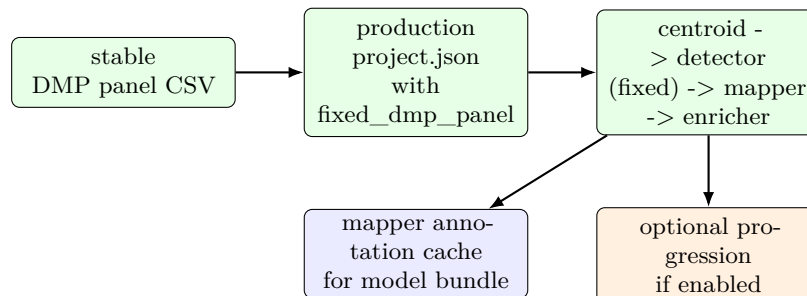


Figure 5: Freeze pipeline mechanics with fixed DMP panel and downstream artifacts.

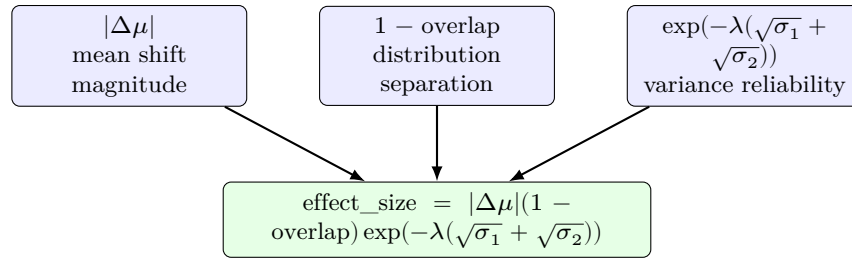


Figure 6: Canonical effect-size decomposition used for biological ranking.

7 Equations and score decomposition figure

8 Limitations and operational caveats

1. Monte Carlo stability improves internal robustness, but external cohort validation remains necessary.
2. Some inferential components are approximations by design (histogram-derived tests, ECDF interpolation grids).
3. Mapper/enricher/progression stages include heuristic ranking and synthesis layers in addition to strict statistical tests.
4. Third-party services (e.g., Enrichr, optional network APIs) can affect reproducibility and runtime behavior.
5. GPU acceleration should be treated as a performance path with fallback safety, not a substitute for explicit environment validation.

9 Rendering

From repo root:

```
quarto render docs/MethylPipeline-analysis.qmd --to pdf
```

Optional HTML rendering:

```
quarto render docs/MethylPipeline-analysis.qmd --to html
```

This document uses TikZ figures authored as raw LaTeX blocks and rendered in the PDF target via lualatex.